

Likelihood-Based Estimation of Jump-Diffusion Processes in Python: The Mathematics Behind the `jump-diffusion-estimation` Library

A pedagogical working paper

Juan David Ospina Arango*

Working draft — July 14, 2026

Abstract

This working paper documents, in a deliberately pedagogical style, the mathematics implemented by the open-source Python library `jump-diffusion-estimation`. The library simulates and estimates discretely sampled jump-diffusion processes whose jump sizes follow a pluggable distribution (normal, skew-normal, skewed generalized error, Kou double-exponential, Student- t). We derive each component in the order a user encounters it: the stochastic differential equation and its Bernoulli (at-most-one-jump) transition density; the jump-size families and the closed-form Gaussian convolutions available for some of them; the FFT-based numerical convolution used when no closed form exists; maximum likelihood estimation with both a local optimizer (L-BFGS-B) and a global one (differential evolution); three uncertainty-quantification routes (profile likelihood, Wald via the observed Fisher information, and the parametric bootstrap); a parametric-bootstrap likelihood-ratio test for the presence of jumps; and goodness-of-fit comparison across jump distributions. Every derivation is followed by the actual code that implements it, together with the design decisions — grid sizes, penalization devices, optimizer defaults — and the reasons behind them. The paper is a companion to the library and to the author’s 2009 thesis, from which several of the numerical schemes are ported.

Keywords: jump-diffusion; maximum likelihood; FFT convolution; differential evolution; profile likelihood; parametric bootstrap; Python.

Contents

1	Introduction	2
1.1	The library at a glance	3
1.2	Notation	4
1.3	How to read this paper	4

*Departamento de Ciencias de la Computación y de la Decisión, Facultad de Minas, Universidad Nacional de Colombia, Medellín, Colombia. jdospina@gmail.com. ORCID: 0000-0002-3547-5247.

2	The model and its transition density	4
2.1	The stochastic differential equation	4
2.2	Exact transition density: the Poisson mixture	5
2.3	The Bernoulli approximation	5
2.4	The code: simulation	5
2.5	The code: the mixture density	6
3	Jump-size distributions	7
3.1	The extension contract	7
3.2	Normal jumps (Merton)	8
3.3	Skew-normal jumps	9
3.4	SGED jumps	10
3.5	Kou's double-exponential jumps	11
3.6	Student- t jumps	12
4	Approximating the convolution by FFT	12
4.1	From integral to circular convolution	13
4.2	The grid	13
4.3	Re-centering and evaluation	14
5	Maximum likelihood estimation	16
5.1	The objective	16
5.2	Starting values from moments	16
5.3	Local optimization: L-BFGS-B	17
5.4	Global optimization: differential evolution	17
5.5	Reported quantities	18
6	Uncertainty quantification: three routes	19
6.1	Wald intervals from the observed information	19
6.2	Profile likelihood intervals	20
6.3	Parametric bootstrap	21
7	Testing for the presence of jumps	22
7.1	Why the textbook calibration fails twice	22
7.2	The parametric bootstrap test	23
8	Comparing jump distributions and validating the estimator	24
8.1	Ranking candidate jump laws	24
8.2	Monte Carlo validation of the estimator	25
9	Closing remarks and research agenda	25

1 Introduction

Financial asset returns exhibit occasional movements far too large to be explained by a Gaussian diffusion. Merton (1976) formalized the now classical remedy: superimpose a compound jump process on the Brownian diffusion, so that ordinary days are driven by the diffusion and

extraordinary days by the jumps. Fitting such a model to discretely sampled data by maximum likelihood is conceptually straightforward — write the transition density of an increment, take logs, sum, maximize — but almost every step hides a numerical or statistical subtlety:

- the transition density involves the *convolution* of the Gaussian diffusion with the jump-size law, which has a closed form only for a few jump distributions;
- the resulting mixture likelihood is *multimodal*, so standard gradient-based optimizers can converge to spurious local optima or stall at box constraints;
- the numerical Hessian at the optimum is frequently ill-conditioned, which makes the textbook (Wald) standard errors fragile;
- testing whether jumps are present at all puts the null hypothesis on the *boundary* of the parameter space while leaving the jump-size parameters *unidentified*, so the classical likelihood-ratio theory does not apply.

The Python library `jump-diffusion-estimation`¹ packages a coherent solution to each of these problems. This working paper explains *how*: every section states the mathematics first and then shows the code that implements it, including the design decisions — discretization grids, penalization devices, optimizer configurations — and the reasoning behind them. Several of those decisions are direct ports of the author’s MSc thesis (Ospina Arango, 2009) and of the subsequent applied work in Ardia et al. (2011); where that is the case we say so explicitly, so the reader can trace each formula to its origin.

1.1 The library at a glance

The package is organized in four cooperating layers, each mapped to a Python subpackage:

Layer	Subpackage	Main classes
Model & simulation	<code>models</code> , <code>simulation</code>	<code>JumpDiffusionModel</code> , <code>JumpDiffusionSimulator</code>
Jump distributions	<code>distributions</code>	<code>JumpDistribution</code> + five concrete families
Estimation & inference	<code>estimation</code>	<code>JumpDiffusionEstimator</code>
Validation & comparison	<code>validation</code>	<code>ValidationExperiment</code> , <code>JumpDistributionComparison</code>

The dependency direction is strictly downward in the table: `JumpDiffusionModel` owns the transition density and path simulation, delegating everything jump-specific to a `JumpDistribution` object; `JumpDiffusionEstimator` evaluates that model’s likelihood at trial parameter vectors and wraps the optimizers and the inference machinery; the validation layer repeatedly drives the estimator on simulated or real data. A user who wants a new jump-size family implements a single class with the distribution’s density — everything else (likelihood, FFT convolution, sampling, estimation, inference, comparison) works unchanged. That extension contract is the subject of Section 3.

¹<https://github.com/jdospina/jump-diffusion-estimation>

1.2 Notation

Throughout the paper, X_t denotes the (log-price) process, μ the drift, $\sigma > 0$ the diffusion volatility, W_t a standard Brownian motion, N_t a Poisson counting process with intensity λ , and J a generic jump size with density $f_J(\cdot; \theta_J)$ and parameter vector θ_J . Observations are n increments $\Delta x_i = x_{t_i} - x_{t_{i-1}}$ on a regular grid with spacing Δt ; in the code, `data` is the array of increments and `dt` is Δt . The Bernoulli jump probability per interval is p (`jump_prob` in the code), which plays the role of $\lambda \Delta t$. The full parameter vector is

$$\vartheta = (\mu, \sigma, p, \theta_J),$$

and the code orders it exactly this way:

```
("mu", "sigma", "jump_prob", *jump_distribution.param_names).
```

We write $\phi(x; m, s^2)$ for the normal density with mean m and variance s^2 , Φ for the standard normal CDF, $*$ for convolution, and $\ell(\vartheta)$ for the log-likelihood. Code identifiers are set in `typewriter` font; all listings are excerpts of the actual library source (docstrings and defensive branches are sometimes elided for readability, never altered).

1.3 How to read this paper

Sections follow the pipeline of an actual analysis. Section 2 derives the model and its Bernoulli-approximated transition density — the single formula the whole library revolves around. Section 3 presents the five jump-size families and their closed-form Gaussian convolutions where they exist. Section 4 derives the FFT convolution scheme used when they do not. Section 5 covers maximum likelihood and the two optimizers. Section 6 derives the three uncertainty-quantification routes, Section 7 the bootstrap test for the presence of jumps, and Section 8 the goodness-of-fit comparison and Monte Carlo validation tools. Section 9 closes with the research agenda this library supports.

2 The model and its transition density

2.1 The stochastic differential equation

The library implements the arithmetic jump-diffusion

$$dX_t = \mu dt + \sigma dW_t + J_t dN_t, \tag{1}$$

where N_t is a Poisson process with intensity λ , independent of W_t , and the jump sizes J_t are i.i.d. draws from $f_J(\cdot; \theta_J)$, independent of both. Because the coefficients are constant, (1) integrates in closed form:

$$X_t = X_0 + \mu t + \sigma W_t + \sum_{k=1}^{N_t} J_k. \tag{2}$$

The arithmetic specification is intended for *log-prices*: if S_t is the price and $X_t = \log S_t$, then (1) is the standard Merton model written on returns, and the increments Δx_i are the log-returns the user hands to the estimator.

Remark 2.1. Equation (2) is exact, not an Euler approximation — constant coefficients make the integral trivial. Every approximation in the library enters later, through the treatment of the jump count per observation interval, not through time discretization of the diffusion.

2.2 Exact transition density: the Poisson mixture

Over an interval of length Δt , the increment implied by (2) is

$$\Delta X = \mu \Delta t + \sigma \sqrt{\Delta t} Z + \sum_{k=1}^K J_k, \quad Z \sim \mathcal{N}(0, 1), \quad K \sim \text{Poisson}(\lambda \Delta t), \quad (3)$$

with Z , K and the J_k mutually independent. Conditioning on $K = k$ and convolving gives the exact transition density as a Poisson mixture:

$$g(\Delta x) = \sum_{k=0}^{\infty} e^{-\lambda \Delta t} \frac{(\lambda \Delta t)^k}{k!} \left(\phi_{\mu \Delta t, \sigma^2 \Delta t} * f_J^{*k} \right) (\Delta x), \quad (4)$$

where f_J^{*k} is the k -fold convolution of the jump density ($f_J^{*0} = \delta_0$). Each additional term requires one more convolution, and for most jump laws none of them is available in closed form.

2.3 The Bernoulli approximation

Design decision 2.1 (At most one jump per interval). For daily data, $\lambda \Delta t$ is small: at $\lambda \Delta t = p$, the probability of two or more jumps in one interval is

$$\mathbb{P}(K \geq 2) = 1 - e^{-p}(1 + p) = \frac{p^2}{2} + O(p^3),$$

i.e. about 0.005 even at the fairly extreme $p = 0.1$. The library therefore truncates (4) at $k \leq 1$ and replaces the Poisson count by a Bernoulli indicator $B \sim \text{Bernoulli}(p)$ with $p = \lambda \Delta t$:

$$\Delta X \approx \mu \Delta t + \sigma \sqrt{\Delta t} Z + B J. \quad (5)$$

This is the same approximation used in [Ospina Arango \(2009\)](#) and [Ardia et al. \(2011\)](#). It buys a two-component mixture density (one convolution instead of infinitely many) at a bias cost of order $(\lambda \Delta t)^2$; quantifying that bias precisely across the $\lambda \Delta t$ range is part of the research agenda of Section 9.

Under (5), the transition density becomes the two-component mixture at the heart of the library,

$$g_{\vartheta}(\Delta x) = (1 - p) \phi(\Delta x; \mu \Delta t, \sigma^2 \Delta t) + p \left(\phi_{\mu \Delta t, \sigma^2 \Delta t} * f_J \right) (\Delta x). \quad (6)$$

Interpreting (6) reads exactly like the data-generating story: with probability $1 - p$ the day is “ordinary” and the increment is Gaussian; with probability p a jump occurred and the increment is the sum of the Gaussian part and one draw from the jump law — hence the convolution.

2.4 The code: simulation

`JumpDiffusionModel.simulate` generates paths directly from (5): a vector of standard normal innovations, a vector of Bernoulli jump indicators, and a vector of jump sizes drawn from the jump distribution, combined step by step.

```
1 dt = T / n_steps
2 brownian_innovations = np.random.normal(0, 1, n_steps)
3 jump_indicators = np.random.binomial(1,
    self.parameters["jump_prob"], n_steps)
```

```

4 jump_sizes = self.jump_distribution.rvs(self._jump_params(),
    size=n_steps)
5
6 for i in range(n_steps):
7     drift = self.parameters["mu"] * dt
8     sigma_term = self.parameters["sigma"] * np.sqrt(dt)
9     diffusion = sigma_term * brownian_innovations[i]
10    jump = jump_indicators[i] * jump_sizes[i]
11    path[i + 1] = path[i] + drift + diffusion + jump

```

Listing 1: Path simulation in `models/jump_diffusion.py` (excerpt).

Note that a jump size is drawn for every step but only *applied* when the indicator fires; this keeps the random-number stream aligned across parameter values, which is convenient for variance-reduction and reproducibility in Monte Carlo work.

2.5 The code: the mixture density

The model’s `_mixture_density` method evaluates (6). The only branch is *how* the convolution term is computed: the jump distribution’s `diffusion_convolved_pdf` is first asked for a closed form (Section 3); if it returns `None`, the generic FFT fallback of Section 4 takes over.

```

1 def _mixture_density(self, x: np.ndarray, dt: float) -> np.ndarray:
2     mu = self.parameters["mu"]
3     sigma = self.parameters["sigma"]
4     p = self.parameters["jump_prob"]
5
6     diffusion_mean = mu * dt
7     diffusion_std = sigma * np.sqrt(dt)
8     diffusion_density = norm.pdf(x, loc=diffusion_mean,
9     scale=diffusion_std)
10
11     # Jump + diffusion component: use a closed form if the jump
12     # distribution has one, otherwise fall back to FFT convolution.
13     jump_params = self._jump_params()
14     jump_diffusion_density =
15         self.jump_distribution.diffusion_convolved_pdf(
16         x, jump_params, diffusion_mean, diffusion_std
17     )
18     if jump_diffusion_density is None:
19         jump_diffusion_density =
20             self.jump_distribution.fft_convolved_pdf(
21             x, jump_params, diffusion_mean, diffusion_std
22         )
23
24     return (1 - p) * diffusion_density + p * jump_diffusion_density

```

Listing 2: The mixture density (6) in `models/jump_diffusion.py`.

The log-likelihood of the observed increments is then simply

$$\ell(\vartheta) = \sum_{i=1}^n \log g_{\vartheta}(\Delta x_i), \quad (7)$$

computed in the model’s `log_likelihood` method with one numerical guard: densities are floored at 10^{-300} before taking logs, so a single increment falling in a region where the numerical density underflows to zero degrades the likelihood instead of destroying it with $\log 0 = -\infty$.

```

1 def log_likelihood(self, data: np.ndarray, dt: float) -> float:
2     increments = data if data.ndim == 1 else np.diff(data)
3     densities = self._mixture_density(increments, dt)
4     densities = np.maximum(densities, 1e-300) # Numerical stability
5     return np.sum(np.log(densities))

```

Listing 3: Log-likelihood (7) with underflow guard.

3 Jump-size distributions

Everything jump-specific in the library lives behind one abstract interface, `JumpDistribution`. This section first describes that contract, then walks through the five concrete families, deriving the closed-form Gaussian convolution wherever one exists — those closed forms are what let the likelihood skip the FFT machinery of Section 4 for the families that admit them.

3.1 The extension contract

A concrete jump distribution must implement four small methods and may override three more:

Method	Status	Role
<code>pdf(x, params)</code>	required	jump-size density $f_J(x; \theta_J)$
<code>default_params()</code>	required	sensible defaults for simulation
<code>param_bounds()</code>	required	optimization box for θ_J
<code>initial_guess(m, s, skew)</code>	required	moment-based starting values
<code>characteristic_scale(params)</code>	optional	scale s_J used to size numerical grids
<code>characteristic_location(params)</code>	optional	center ℓ_J used to place numerical grids
<code>diffusion_convolved_pdf(...)</code>	optional	closed form for $\phi * f_J$, if known
<code>rvs(params, size)</code>	optional	native sampler, if available
<code>mean(params), variance(params)</code>	generic	numerical $\mathbb{E}[J]$, $\text{Var}[J]$

Design decision 3.1 (Density-only extensibility). The base class provides generic numerical fallbacks for the two hard operations: `fft_convolved_pdf` approximates $\phi * f_J$ by FFT (Section 4), and `rvs` samples by numerically inverting the CDF obtained from the cumulative sum of the discretized density. A new distribution therefore only *needs* its `pdf`; closed-form overrides are pure speed-ups. This is what makes the library’s “pluggable jump law” promise cheap to keep.

The numerical fallbacks share one discretization of the density, `_moment_grid`: $2m = 2^{13}$ half-integer-offset points (coarser than the FFT grid — ample for sampling and moments) with step $h = s_J/200$, centered on the law’s characteristic location ℓ_J so that a shifted distribution (a Student- t or SGED with a large `jump_loc`) is not sampled from an empty grid. On top of it, the generic `rvs` inverts the cumulative sum by linear interpolation, and the generic `mean/variance` integrate the discretized density — the moments that feed the model-implied increment diagnostics of Section 6, verified in the test suite against the closed-form moments of the normal, skew-normal, Kou and located Student- t laws:

```

1 def _moment_grid(self, params):
2     jump_std = self.characteristic_scale(params)
3     h = jump_std / 200.0
4     m = 2**12 # a coarser grid than fft_convolved_pdf suffices here
5     k = np.arange(-m + 0.5, m, 1.0)
6     # Centered on the distribution's location so that a shifted
7     # distribution (large jump_loc) does not fall off the grid.
8     x_grid = self.characteristic_location(params) + k * h
9     density = np.maximum(self.pdf(x_grid, params), 0.0)
10    return x_grid, density, h
11
12 def rvs(self, params, size, random_state=None):
13     x_grid, density, h = self._moment_grid(params)
14     cdf = np.cumsum(density) * h
15     cdf /= cdf[-1]
16     u = rand.uniform(0.0, 1.0, size=size)
17     return np.interp(u, cdf, x_grid)
18
19 def mean(self, params):
20     x_grid, density, h = self._moment_grid(params)
21     total = float(np.sum(density) * h)
22     if total <= 0:
23         return float("nan")
24     return float(np.sum(x_grid * density) * h / total)

```

Listing 4: Shared grid, inverse-CDF sampling and numerical moments in `distributions/base.py` (excerpts).

The two `characteristic_*` hooks deserve a word: the numerical grids must be sized by how spread out the jump law is and placed where its mass actually lives. The default `characteristic_scale` reads the parameter named `jump_scale` (the convention of the normal, skew-normal, SGED and Student-*t* families); distributions parameterized differently override it — Kou’s law, with separate up and down scales, returns the larger of the two. The default `characteristic_location` reads `jump_loc`, falling back to 0 for the families centered at the origin; laws whose center is not a parameter but stays within a few scales of zero (Kou again) are covered by the grids’ tail margins and need no override.

3.2 Normal jumps (Merton)

The classical model of Merton (1976): $J \sim \mathcal{N}(0, s^2)$ with a single parameter `jump_scale = s`. Its convolution with the diffusion is the textbook result that a sum of independent normals is normal:

$$\phi_{m_d, \sigma_d^2} * f_J = \phi(\cdot; m_d, \sigma_d^2 + s^2), \quad (8)$$

where $m_d = \mu\Delta t$ and $\sigma_d = \sigma\sqrt{\Delta t}$ are the diffusion mean and standard deviation of Section 2.

```

1 def diffusion_convolved_pdf(self, x, params, diffusion_mean,
2     diffusion_std):
3     combined_std = np.sqrt(diffusion_std**2 + params["jump_scale"]
4         ** 2)
5     return norm.pdf(x, loc=diffusion_mean, scale=combined_std)

```

Listing 5: Closed-form convolution (8) in `distributions/normal.py`.

Beyond historical importance, `NormalJump` plays a structural role: it is nested inside the skew-normal family (at zero skewness) and inside the SGED (at $\nu = 2$, $\xi = 1$), which makes it the natural null model when asking whether asymmetry or heavy tails are really needed (Section 8).

3.3 Skew-normal jumps

The skew-normal distribution of [Azzalini \(1985\)](#) with location 0, scale ω (`jump_scale`) and shape α (`jump_skew`) has density

$$f_J(x) = \frac{2}{\omega} \phi\left(\frac{x}{\omega}\right) \Phi\left(\alpha \frac{x}{\omega}\right). \quad (9)$$

Positive α skews the jumps to the right, negative to the left, and $\alpha = 0$ recovers the normal.

The family is closed under convolution with an independent normal ([Azzalini, 1986](#))²: if $J \sim \text{SN}(0, \omega, \alpha)$ and $D \sim \mathcal{N}(m_d, \sigma_d^2)$ independently, then $D + J$ is again skew-normal with location m_d , scale $\tilde{\omega} = \sqrt{\omega^2 + \sigma_d^2}$, and a *new* shape parameter obtained by passing through the δ -parameterization:

$$\delta = \frac{\alpha}{\sqrt{1 + \alpha^2}}, \quad \tilde{\delta} = \frac{\omega \delta}{\tilde{\omega}}, \quad \tilde{\alpha} = \frac{\tilde{\delta}}{\sqrt{1 - \tilde{\delta}^2}}. \quad (10)$$

Design decision 3.2 (Go through δ , not α). It is tempting to guess that the new shape is simply $\alpha\omega/\tilde{\omega}$ — rescale the shape the way the scale was rescaled. That shortcut is *wrong*: it is off by about 0.2% at $\alpha = 2$, an error easily mistaken for FFT discretization noise but which biases the likelihood systematically. The library’s implementation was verified against direct numerical convolution, and the code carries a comment warning future maintainers away from the shortcut.

```

1 def diffusion_convolved_pdf(self, x, params, diffusion_mean,
2   diffusion_std):
3     omega = params["jump_scale"]
4     alpha = params["jump_skew"]
5     combined_std = np.sqrt(diffusion_std**2 + omega**2)
6
7     # Azzalini & Henze (1986): the new shape parameter is obtained by
8     # going through the "delta" parameterization, not
9     # alpha*omega/std.
10    delta = alpha / np.sqrt(1 + alpha**2)
11    adjusted_delta = omega * delta / combined_std
12    adjusted_skew = adjusted_delta / np.sqrt(1 - adjusted_delta**2)
13
14    return skewnorm.pdf(x, a=adjusted_skew, loc=diffusion_mean,
15                        scale=combined_std)

```

Listing 6: Skew-normal convolution (10) in `distributions/skew_normal.py`.

²The library’s code comments cite this result as “Azzalini & Henze (1986)”; the closure property itself appears in [Azzalini \(1986\)](#), while Henze’s 1986 paper gives the related probabilistic representation of the skew-normal.

3.4 SGED jumps

The skewed generalized error distribution is the library’s most flexible symmetric-plus-skewness family and the centerpiece of [Ospina Arango \(2009\)](#); see also [Theodossiou \(2015\)](#) for its role in finance. It is built in three steps, and the code mirrors the construction function by function.

Step 1: the GED base. The generalized error distribution with shape $\nu > 0$, standardized to mean 0 and variance 1, has density

$$f_{\text{GED}}(z; \nu) = \frac{\nu \exp\left(-\frac{1}{2} |z/\lambda_\nu|^\nu\right)}{\lambda_\nu 2^{1+1/\nu} \Gamma(1/\nu)}, \quad \lambda_\nu = \sqrt{\frac{2^{-2/\nu} \Gamma(1/\nu)}{\Gamma(3/\nu)}}. \quad (11)$$

$\nu = 2$ gives the standard normal, $\nu = 1$ the Laplace; smaller ν means heavier tails.

Step 2: skewing à la Fernández–Steel. [Fernández and Steel \(1998\)](#) skew any symmetric density by scaling its two halves differently with a parameter $\xi > 0$:

$$f_\xi(z; \nu) = \frac{2}{\xi + 1/\xi} f_{\text{GED}}\left(\xi^{-\text{sign}(z)} z; \nu\right). \quad (12)$$

$\xi > 1$ stretches the right tail (positive skew), $\xi < 1$ the left; $\xi = 1$ recovers symmetry. Skewing shifts the mean and changes the variance, so (12) is not yet a location-scale-friendly object.

Step 3: re-standardization. With $\lambda = \lambda_\nu$, the absolute moments of the GED give

$$m_1 = \frac{\Gamma(2/\nu) \lambda 2^{1/\nu}}{\Gamma(1/\nu)}, \quad m_2 = \frac{\Gamma(3/\nu) \lambda^2 2^{2/\nu}}{\Gamma(1/\nu)}, \quad (13)$$

from which the mean and variance of the skewed density (12) are

$$\mu_\xi = \left(\xi - \frac{1}{\xi}\right) m_1, \quad \sigma_\xi^2 = \left(\xi^2 + \frac{1}{\xi^2}\right) (m_2 - m_1^2) + 2m_1^2 - m_2. \quad (14)$$

The standardized SGED — mean 0, variance 1, the form used throughout the thesis and the `fGarch` R package — is then

$$f_{\text{SGED}}^*(z; \nu, \xi) = \sigma_\xi f_\xi(\sigma_\xi z + \mu_\xi; \nu), \quad (15)$$

and the four-parameter jump law with location `jump_loc` = ℓ and scale `jump_scale` = s (which is the standard deviation) is $f_J(x) = f_{\text{SGED}}^*((x - \ell)/s; \nu, \xi)/s$.

```

1 def _ged_lambda(nu):
2     return np.sqrt(2 ** (-2 / nu) * gamma(1 / nu) / gamma(3 / nu))
3
4 def _ged_pdf(z, nu):
5     # eq. (GED): standardized
6     base
7     lam = _ged_lambda(nu)
8     return (nu * np.exp(-0.5 * np.abs(z / lam) ** nu)
9             / (lam * 2 ** (1 + 1 / nu) * gamma(1 / nu)))
10
11 def _fernandez_steel_pdf(z, nu, xi):
12     # eq. (FS): two-piece
13     skewing

```

```

10     sign = np.sign(z)
11     return (2.0 / (xi + 1.0 / xi)) * _ged_pdf(np.power(xi, -sign) *
12         z, nu)
13 def _skew_moments(nu, xi):                # eqs. (m1,m2) and (mu_xi,
14     sigma_xi)
15     lam = _ged_lambda(nu)
16     m1 = gamma(2 / nu) * lam * 2 ** (1 / nu) / gamma(1 / nu)
17     m2 = gamma(3 / nu) * lam**2 * 2 ** (2 / nu) / gamma(1 / nu)
18     mu_xi = (xi - 1 / xi) * m1
19     sigma_xi2 = (xi**2 + 1 / xi**2) * (m2 - m1**2) + 2 * m1**2 - m2
20     return mu_xi, np.sqrt(sigma_xi2)
21 def _standardized_sged_pdf(z, nu, xi):    # eq. (SGED): mean 0, var 1
22     mu_xi, sigma_xi = _skew_moments(nu, xi)
23     return sigma_xi * _fernandez_steel_pdf(sigma_xi * z + mu_xi, nu,
24         xi)

```

Listing 7: The three-step SGED construction in `distributions/sged.py`.

No closed form is known for $\phi * f_{\text{SGED}}$, so this family always goes through the FFT convolution of Section 4. Its special cases ($\nu=2, \xi=1 \Rightarrow$ normal; $\nu=1, \xi=1 \Rightarrow$ Laplace) are verified against closed forms in the test suite — a pattern used throughout the library: every numerical fallback is tested where an exact answer exists.

3.5 Kou’s double-exponential jumps

Kou (2002) models jumps as a mixture of two one-sided exponentials: with probability q (`jump_prob_up`) the jump is positive, $J \sim \text{Exp}(\eta_u)$ with mean η_u (`jump_scale_up`); otherwise it is negative with magnitude $\sim \text{Exp}(\eta_d)$ (`jump_scale_down`):

$$f_J(x) = q \frac{1}{\eta_u} e^{-x/\eta_u} \mathbf{1}\{x > 0\} + (1 - q) \frac{1}{\eta_d} e^{x/\eta_d} \mathbf{1}\{x < 0\}. \quad (16)$$

Asymmetry here comes from $q \neq \frac{1}{2}$ or $\eta_u \neq \eta_d$, not from a shape parameter.

The Gaussian convolution again has a closed form, through the *exponentially modified Gaussian* (ex-Gaussian): if $Y \sim \text{Exp}(\eta)$ and $D \sim \mathcal{N}(m_d, \sigma_d^2)$, the density of $D + Y$ is

$$f_{D+Y}(x) = \frac{1}{2\eta} \exp\left(\frac{\sigma_d^2}{2\eta^2} - \frac{x - m_d}{\eta}\right) \text{erfc}\left(\frac{1}{\sqrt{2}} \left(\frac{\sigma_d}{\eta} - \frac{x - m_d}{\sigma_d}\right)\right), \quad (17)$$

and the downward branch follows by the reflection $x \mapsto -x$. The convolution of (16) with the diffusion is the q -weighted mixture of one direct and one reflected ex-Gaussian.

Design decision 3.3 (Reuse `scipy.stats.exponnorm`, do not re-derive). Formula (17) is numerically treacherous: the exponential factor overflows exactly where the `erfc` factor underflows, and the stable evaluation requires the scaled complementary error function `erfcx`. SciPy’s `exponnorm` distribution already implements exactly this (parameterized by $K = \eta/\sigma_d$), so the library delegates to it rather than re-implementing the stability tricks — and verifies the result against direct numerical convolution in the tests.

```

1 def diffusion_convolved_pdf(self, x, params, diffusion_mean,
    diffusion_std):
2     p = params["jump_prob_up"]
3     up = p * exponnorm.pdf(
4         x, K=params["jump_scale_up"] / diffusion_std,
5         loc=diffusion_mean, scale=diffusion_std)
6     down = (1 - p) * exponnorm.pdf(
7         -x, K=params["jump_scale_down"] / diffusion_std,
8         loc=-diffusion_mean, scale=diffusion_std)
9     return up + down

```

Listing 8: Kou convolution as a mixture of ex-Gaussians in `distributions/kou.py`.

Note the downward branch: reflecting the density requires negating *both* the evaluation points and the location of the Gaussian component, which is why the code passes `-x` and `loc=-diffusion_mean`.

3.6 Student- t jumps

The heavy-tailed symmetric alternative: a location-scale Student- t with degrees of freedom ν_t (`jump_df`), location `jump_loc` and scale `jump_scale`.

Design decision 3.4 (Standardize the scale to the standard deviation). SciPy’s raw `t` scale s_{scipy} relates to the standard deviation through the degrees of freedom: $\text{sd} = s_{\text{scipy}} \sqrt{\nu_t / (\nu_t - 2)}$ for $\nu_t > 2$. If the raw scale were exposed as the parameter, the fitted “scale” would not be comparable across candidate jump laws, and ν_t would entangle spread with tail weight. The library therefore re-parameterizes so that `jump_scale` is the standard deviation,

$$s_{\text{scipy}} = s \sqrt{\frac{\nu_t - 2}{\nu_t}}, \quad (18)$$

the same standardized- t convention used for GARCH- t innovations by [Bollerslev \(1987\)](#). The lower bound of `jump_df` is 2.05, keeping the variance finite so that (18) is well defined.

No closed form exists for $\phi * t_{\nu_t}$, so like the SGED this family relies on the FFT fallback — which is the subject of the next section.

4 Approximating the convolution by FFT

When the jump family has no closed-form Gaussian convolution (SGED, Student- t , and any user-added law that implements only `pdf`), the mixture density (6) needs the convolution

$$(\phi_d * f_J)(x) = \int_{-\infty}^{\infty} \phi_d(t) f_J(x - t) dt, \quad \phi_d \equiv \phi(\cdot; m_d, \sigma_d^2), \quad (19)$$

evaluated at every observed increment, thousands of times per likelihood maximization. The library approximates it with the discrete Fourier transform, porting the scheme of [Ospina Arango \(2009, ch. 4\)](#) (the `f.rec` routine, originally in R) to NumPy. This section derives the scheme and each of its tuning constants.

4.1 From integral to circular convolution

Discretize (19) by the rectangle rule on a uniform grid of spacing h : writing $x_k = kh$ for grid points,

$$(\phi_d * f_J)(x_n) \approx h \sum_k \phi_d(x_k) f_J(x_n - x_k), \quad (20)$$

which is h times the *discrete* (linear) convolution of the two sampled densities. The FFT computes circular, not linear, convolution: for two length- N vectors a and b ,

$$\text{IDFT}(\text{DFT}(a) \cdot \text{DFT}(b))_n = \sum_{k=0}^{N-1} a_k b_{(n-k) \bmod N}, \quad (21)$$

at cost $O(N \log N)$ instead of $O(N^2)$. Circularity means the result wraps around: mass that the linear convolution would place beyond the grid ends re-enters at the opposite end. The scheme therefore only works if the grid is wide enough that both densities are numerically negligible near its ends — then the wrapped-around contamination is negligible too. This is the requirement that drives the grid-size decisions below.

Remark 4.1 (NumPy vs. R normalization). Equation (21) holds for NumPy’s convention, where `np.fft.ifft` divides by N . R’s `fft(..., inverse = TRUE)` does *not* divide by N , so the thesis’ original R code carried an explicit $1/N$ that must *not* be ported. The code comments this trap explicitly — an example of a design decision that exists purely to protect future maintainers.

4.2 The grid

The library samples both densities at $N = 2m$ points with *half-integer offsets*,

$$x_k = \left(k + \frac{1}{2}\right)h, \quad k = -m, -m+1, \dots, m-1, \quad m = 2^j, \quad (22)$$

i.e. a symmetric grid from $-(m - \frac{1}{2})h$ to $(m - \frac{1}{2})h$ that straddles zero without containing it.

Design decision 4.1 (Half-integer offsets). Two properties motivate the offset grid. First, symmetry: the grid covers $[-mh, mh]$ evenly with no duplicated or privileged center point, so symmetric densities are sampled symmetrically. Second, it dodges the center singularity: some candidate jump densities are unbounded at their mode (the Laplace corner at 0, or the SGED with $\nu < 1$), and evaluating exactly at 0 would put one enormous (or infinite) sample into the discrete convolution. With half-integer offsets every evaluation point is at distance $\geq h/2$ from zero. The same grid construction is reused by the generic sampler of Section 3.1.

Design decision 4.2 (Resolution first, coverage as a binding guard). The step h must resolve the *narrower* of the two densities being convolved — oversampling the wide one costs nothing, undersampling the narrow one destroys the approximation. Hence the base heuristic

$$h = \frac{\min(\sigma_d, s_J)}{200},$$

where s_J is the jump law’s `characteristic_scale`: two hundred points per standard deviation of the narrowest component, with $j = 15$ giving $N = 2^{16} = 65,536$ grid points. Both constants are ported from Ospina Arango (2009) and exposed as arguments (`j`, `h`).

Resolution alone, however, does not guarantee *coverage*. The half-width bought by the base heuristic is $mh \geq 163$ standard deviations of the *narrower* density — which says nothing about

the *wider* one. An optimizer legitimately probes candidates where the jump scale dwarfs the diffusion scale (ratio 160:1 leaves the jump density with essentially zero mass on a grid sized by σ_d), or where the jump location ℓ_J (the law’s `characteristic_location`, Section 3.1) sits far from the origin. The step is therefore *coarsened* whenever the span could not contain the mass of both densities:

$$mh < \underbrace{|m_d| + |\ell_J| + 10(\sigma_d + s_J)}_{\text{half-span needed}} \implies h = \frac{|m_d| + |\ell_J| + 10(\sigma_d + s_J)}{m}. \quad (23)$$

A slightly coarser grid that preserves total mass beats a fine grid that silently truncates it. In the default daily-data regime (scale ratios up to ≈ 18) the guard never binds, so ordinary fits are unaffected; it exists for the extreme candidates that global optimizers and boundary studies visit.

4.3 Re-centering and evaluation

One subtlety remains. The circular convolution (21) indexes vectors by array position $0, \dots, N-1$, while our samples live on the physical grid (22). If entry i of each input vector holds the density at $x_i = (i - m + \frac{1}{2})h$, then output entry n of (21) accumulates products of samples whose physical locations add to

$$x_i + x_k = (i + k - 2m + 1)h,$$

with $i + k \equiv n \pmod{N}$. Matching this back to the target grid (22) shows the output is displaced by half the array length: entry n of the raw FFT result corresponds to physical point x_{n-m} (modulo N), not x_n . Swapping the two halves of the output array — a circular shift by m , NumPy’s `fftshift` — restores alignment, after which entry n approximates $(\phi_d * f_J)(x_n)$ on the same symmetric grid the inputs were sampled on.

Finally, the convolved density is only known on grid points, and the likelihood needs it at the observed increments: the code linearly interpolates and clips tiny negative values produced by floating-point cancellation to zero.

Design decision 4.3 (Out-of-grid points evaluate to zero — individually). Once the coverage guard (23) ensures the grid contains the densities’ mass, the true convolved density beyond the grid ends is genuinely negligible — so any observed increment falling there is assigned density 0 (`np.interp` with `left=0.0`, `right=0.0`), which the likelihood’s underflow floor (Section 2) then turns into a strong but finite penalty on that parameter vector.

The qualifier *individually* earns its place in the title: an earlier version of this routine returned an all-zeros vector as soon as *any single* requested point fell outside the grid. The 2026 audit of the library showed the consequence: one extreme observation (e.g. a log-return of 0.11 in the S&P 500 sample) silently zeroed the whole convolution term for *every* candidate whose jump scale was small enough that the grid, sized by Decision 4.2, did not reach that observation — producing an artificially flat, floored likelihood over the entire small-scale region. That cliff biased estimation against small jump scales, broke the smoothness L-BFGS-B relies on, and would have contaminated precisely the $p \rightarrow 0$ / scale $\rightarrow 0$ boundary regions that the inference and testing machinery of Sections 6–7 cares most about. The lesson generalizes: a numerical guard must fail *pointwise*, never collectively.

```

1 def fft_convolved_pdf(self, x, params, diffusion_mean, diffusion_std,
2                           j=15, h=None):
3     x = np.asarray(x, dtype=float)
```

```

4     jump_std = self.characteristic_scale(params)
5     if jump_std is None or jump_std <= 0:
6         return np.zeros_like(x)
7     jump_loc = self.characteristic_location(params)
8
9     m = 2**j
10    if h is not None:
11        step = h
12    else:
13        # Step chosen for resolution, then coarsened if the span
14        could
15        # not contain the mass of both densities (plus tail margins).
16        step = min(diffusion_std, jump_std) / 200.0
17        half_span_needed = (
18            abs(diffusion_mean) + abs(jump_loc) + 10.0 *
19            (diffusion_std + jump_std)
20        )
21        if m * step < half_span_needed:
22            step = half_span_needed / m
23
24    k = np.arange(-m + 0.5, m, 1.0)    # length 2*m, half-integer
25    offsets
26    x_grid = k * step
27
28    f_diffusion = norm.pdf(x_grid, loc=diffusion_mean,
29                           scale=diffusion_std)
30    f_jump = self.pdf(x_grid, params)
31    if not (np.all(np.isfinite(f_diffusion)) and
32            np.all(np.isfinite(f_jump))):
33        return np.zeros_like(x)
34
35    # numpy's ifft already divides by n, unlike R's fft(...,
36    inverse=TRUE)
37    conv = np.real(np.fft.ifft(np.fft.fft(f_diffusion) *
38                                   np.fft.fft(f_jump)))
39    conv *= step    # rectangle-rule
40    weight
41    conv = np.concatenate([conv[m:], conv[:m]])    # re-center around
42    x_grid
43    conv = np.maximum(conv, 0.0)
44
45    return np.interp(x, x_grid, conv, left=0.0, right=0.0)

```

Listing 9: The full FFT convolution in distributions/base.py.

Line by line against the derivation: the grid is (22); the step selection implements Decision 4.2 with the coverage guard (23); the product of FFTs and single inverse FFT is (21) (with the normalization of Remark 4.1); the multiplication by `step` is the rectangle-rule weight of (20); the `concatenate` is the circular shift by m ; and the final `interp`, with its pointwise zeros outside the grid (Decision 4.3), evaluates the result at the observed increments.

Remark 4.2 (Accuracy checks). The scheme is validated in the test suite by running it on families whose convolution is known — normal and skew-normal — and comparing against (8) and (10),

as well as by checking the SGED special cases of Section 3.4. This gives end-to-end assurance without ever trusting the FFT code on faith for the families where no closed form exists.

5 Maximum likelihood estimation

With the transition density in hand, estimation is the maximization of (7) over $\vartheta = (\mu, \sigma, p, \theta_J)$. The estimator class, `JumpDiffusionEstimator`, wraps this in a numerically defensive objective and offers two optimizers whose contrast is one of the library’s central themes: a fast local method that needs a good start, and a global method that needs none.

5.1 The objective

Optimizers minimize, so the objective is the *negative* log-likelihood, with two guard layers wrapped around (7):

```

1 def log_likelihood(self, params: np.ndarray) -> float:
2     sigma, jump_prob = params[1], params[2]
3
4     if sigma <= 0:
5         return np.inf
6     if jump_prob < 0 or jump_prob > 1:
7         return np.inf
8
9     self._model.update_parameters(**dict(zip(self._param_names,
10                                              params)))
11     value = self._model.log_likelihood(self.increments, self.dt)
12     # Penalize non-numeric likelihood values so that population-based
13     # optimizers simply discard the offending candidates -- same
14     # device
15     # as in Ospina Arango (2009).
16     if not np.isfinite(value):
17         return np.inf
18     return -value

```

Listing 10: The objective in `estimation/maximum_likelihood.py`.

Design decision 5.1 (Map every pathology to $+\infty$). Invalid parameters ($\sigma \leq 0$, $p \notin [0, 1]$) and non-finite likelihood evaluations (numerical accidents deep in the FFT or in extreme tails) all return $+\infty$. For a population-based optimizer this is exactly the right poison: an infinite objective makes the candidate lose every selection comparison and disappear from the population without corrupting the ranking of the finite candidates. The same device — mapping NaN evaluations to a large positive constant — was used in [Ospina Arango \(2009\)](#); $+\infty$ is its cleanest float representation.

5.2 Starting values from moments

Gradient methods need a starting point. The estimator builds one from the sample moments of the increments: since $\mathbb{E}[\Delta X] = \mu \Delta t + p \mathbb{E}[J]$ and $\text{Var}[\Delta X] = \sigma^2 \Delta t + p \text{Var}[J] + p(1 - p) \mathbb{E}[J]^2$,

ignoring the (typically small) jump contributions gives the first-pass guesses

$$\hat{\mu}_0 = \frac{\overline{\Delta x}}{\Delta t}, \quad \hat{\sigma}_0 = \frac{s_{\Delta x}}{\sqrt{\Delta t}}, \quad \hat{p}_0 = 0.1, \quad (24)$$

and each jump family contributes its own heuristic through its `initial_guess` method — e.g. the skew-normal starts `jump_scale` at half the increment standard deviation and signs `jump_skew` by the sample skewness. These are deliberately crude: their job is to land in the right order of magnitude, not to be accurate. When they are not enough — and Section 5.4 shows this happens in practice — the answer is a global optimizer, not a cleverer guess.

5.3 Local optimization: L-BFGS-B

The default method hands the objective to `scipy.optimize.minimize` with `method="L-BFGS-B"`, box constraints from the model ($\sigma > 0$, $0 < p < 1$, plus each jump parameter’s bounds from Section 3.1), and tolerances `maxiter=1000`, `ftol=1e-9`. On well-behaved likelihoods (normal or skew-normal jumps, moderate p) it converges in milliseconds. Its failure modes on this problem are well documented in Ospina Arango (2009): with heavy-tailed or strongly skewed jump laws the mixture likelihood develops multiple local optima and long, curved ridges, and a quasi-Newton iterate either stalls on a box boundary or polishes whichever local optimum captures it.

5.4 Global optimization: differential evolution

Differential evolution (Storn and Price, 1997) maintains a population of NP candidate vectors and evolves it by mutation, crossover and greedy selection. One generation of the *rand/1/bin* strategy — the one used here — transforms each population member x_i as follows:

$$\text{mutation: } v_i = x_{r_1} + F(x_{r_2} - x_{r_3}), \quad r_1 \neq r_2 \neq r_3 \neq i \text{ random}, \quad (25)$$

$$\text{crossover: } u_{i,d} = \begin{cases} v_{i,d} & \text{if } U_d \leq \text{CR or } d = d_{\text{rand}}, \\ x_{i,d} & \text{otherwise,} \end{cases} \quad (26)$$

$$\text{selection: } x_i \leftarrow u_i \text{ iff } f(u_i) \leq f(x_i). \quad (27)$$

Mutation (25) perturbs a random member by a scaled difference of two others, so step sizes adapt to the population’s own spread in each direction — large early on, shrinking as the population contracts onto a basin. Binomial crossover (26) mixes mutant and parent coordinate-wise. Selection (27) is greedy but elitist: the best-ever point can never be lost. No gradients appear anywhere, so multimodality, ridges, and the occasional $+\infty$ from Decision 5.1 are all handled gracefully.

Design decision 5.2 (Port the thesis’ DEoptim configuration). The applied finding of Ospina Arango (2009) — refined in Ardia et al. (2011) — is that DE with R’s DEoptim defaults (strategy *rand/1*, NP = 70, 400 generations) recovers the SGED jump-diffusion optimum even from very wide bounds, where L-BFGS-B and simulated annealing both fail. The library ports this configuration to `scipy.optimize.differential_evolution`:

- `strategy="rand1bin"` is exactly *rand/1* with binomial crossover;
- SciPy sizes the population as `popsizexk` where k is the number of parameters, so `popsizex10` reproduces NP = 70 for the 7-parameter SGED model and scales proportionally for other jump families;

- `maxiter=400` matches the generation cap, and SciPy’s convergence tolerance (`tol=0.01`, stop when the population’s objective spread falls below 1% of its mean) implements the early-stopping criterion the thesis recommends — in practice runs stop well before 400 generations.

Design decision 5.3 (Finite, data-driven search box). DE samples its initial population uniformly over the bounds, so the half-open boxes used by L-BFGS-B (μ unbounded, $\sigma < \infty$) must be replaced by finite ones. The library builds them from the data: μ ranges over $\hat{\mu}_0 \pm \max(1, 10|\hat{\mu}_0|, 10\hat{\sigma}_0)$, σ over $(10^{-6}, 10\hat{\sigma}_0)$, and any unbounded jump parameter is capped at $\pm 20 s_{\Delta x}$ — a single jump twenty times the typical increment is a generous ceiling, especially since $s_{\Delta x}$ is itself inflated by whatever jumps the data contains. Wide boxes are deliberate: the thesis’ point is precisely that DE does not need tight prior knowledge, so the defaults encode ignorance, not information. Bounds the user supplies, and finite bounds from the jump family, are kept as-is.

```

1 if bounds is None:
2     bounds = self._finite_bounds()
3 lows, highs = zip(*bounds)
4 de_bounds = Bounds(np.asarray(lows, float), np.asarray(highs, float))
5
6 de_kwargs = {
7     "strategy": "rand1bin",    # DEoptim's rand/1 with binomial
8                               # crossover
9     "maxiter": 400,           # thesis generation cap
10    "popsize": 10,             # 10 * 7 params = NP 70 for the SGED
11                               # model
12 }
13 de_kwargs.update(kwargs)      # user overrides, e.g. seed=, workers=
14 return differential_evolution(func=self.log_likelihood,
15                               bounds=de_bounds, **de_kwargs)

```

Listing 11: DE configuration in `estimation/maximum_likelihood.py` (excerpt).

The cost asymmetry is worth stating plainly: L-BFGS-B typically needs tens-to-hundreds of likelihood evaluations, DE thousands — seconds instead of milliseconds with closed-form convolutions, and proportionally more through the FFT path. The recommended workflow is therefore to fit with DE (optionally passing `seed=` for reproducibility and `workers=-1` to parallelize the population evaluation) and reserve L-BFGS-B for quick exploratory fits and for the repeated re-fits inside bootstrap loops, where the data are simulated near a known optimum.

5.5 Reported quantities

`estimate()` returns the fitted parameters, the maximized log-likelihood $\ell(\hat{\vartheta})$, and the information criteria

$$\text{AIC} = 2k - 2\ell(\hat{\vartheta}), \quad \text{BIC} = k \log n - 2\ell(\hat{\vartheta}), \quad (28)$$

with $k = \dim \vartheta$ and n the number of increments — the ingredients for the model comparison of Section 8 — plus the raw optimizer diagnostics and a convergence flag that downstream bootstrap loops use to discard failed re-fits.

6 Uncertainty quantification: three routes

A point estimate without standard errors is half an answer. The library implements three inference routes with deliberately different assumptions and costs, summarized in Table 1. Offering all three is itself a design decision: comparing their finite-sample coverage across jump families is one of the research questions this library was built to answer (Section 9).

Route (method)	Assumes	Cost
Wald <code>estimate_wald_standard_errors</code>	asymptotic normality, regular MLE	one Hessian
Profile <code>estimate_standard_errors</code>	Wilks' χ^2 asymptotics	re-optimizations
Bootstrap <code>estimate_bootstrap_standard_errors</code>	correct model specification	B full re-fits

Table 1: The three inference routes. Cost grows downward; robustness to boundary and non-quadratic likelihoods grows the same way.

6.1 Wald intervals from the observed information

Classical first-order asymptotics: under regularity conditions, $\hat{\vartheta} \approx \mathcal{N}(\vartheta_0, I(\hat{\vartheta})^{-1})$ where

$$I(\hat{\vartheta}) = -\nabla^2 \ell(\hat{\vartheta}) \quad (29)$$

is the *observed* Fisher information — the Hessian of the negative log-likelihood at the estimate. Standard errors are the square roots of the diagonal of I^{-1} , and intervals are $\hat{\theta}_r \pm z_{1-\alpha/2} \widehat{\text{SE}}_r$.

Design decision 6.1 (Observed, not expected, information). Following Efron and Hinkley (1978), the library uses the observed information rather than the expected Fisher information: for this mixture likelihood the expectation has no closed form and would itself require numerical integration, and Efron and Hinkley argue the observed information is the more relevant conditioning quantity anyway.

Since the code's objective is already the negative log-likelihood, its Hessian is exactly (29), built by central finite differences: on the diagonal

$$\frac{\partial^2 f}{\partial \theta_r^2} \approx \frac{f(\hat{\vartheta} + h_r e_r) - 2f(\hat{\vartheta}) + f(\hat{\vartheta} - h_r e_r)}{h_r^2}, \quad (30)$$

and off the diagonal the four-point mixed formula

$$\frac{\partial^2 f}{\partial \theta_r \partial \theta_s} \approx \frac{f_{++} - f_{+-} - f_{-+} + f_{--}}{4h_r h_s}, \quad (31)$$

where $f_{\pm\pm} = f(\hat{\vartheta} \pm h_r e_r \pm h_s e_s)$.

Design decision 6.2 (Bound-aware step sizes). The step for parameter r is $h_r = 10^{-4} \max(|\hat{\theta}_r|, 10^{-2})$ — relative, with an absolute floor so parameters estimated near zero (drift, jump location) still get a usable step. Crucially, each step is then shrunk to at most a quarter of the distance to the nearest bound: the objective returns $+\infty$ outside the bounds (Decision 5.1), and a single

infinite evaluation would destroy the difference quotients (30)–(31). Entries that still cannot be evaluated (a parameter pinned exactly to a bound) are recorded as `NaN`, and the inversion is guarded: if the Hessian is non-finite or singular, every Wald SE is reported as `NaN` rather than silently wrong. This is not an edge case — $\hat{p}$ near 0 or a jump scale collapsing toward its lower bound are exactly the interesting regimes.

6.2 Profile likelihood intervals

The Wald construction forces symmetric intervals and trusts a quadratic approximation of the log-likelihood. The profile likelihood does neither. For a scalar parameter of interest θ_r , define

$$\ell_p(v) = \max_{\vartheta: \theta_r = v} \ell(\vartheta), \quad (32)$$

the log-likelihood maximized over all *other* parameters with θ_r pinned at v . By Wilks’ theorem (Wilks, 1938), under the true value $2[\ell(\hat{\vartheta}) - \ell_p(\theta_{r,0})] \rightsquigarrow \chi_1^2$, so

$$\text{CI}_{1-\alpha} = \left\{ v : \ell_p(v) \geq \underbrace{\ell(\hat{\vartheta}) - \frac{1}{2}\chi_{1,1-\alpha}^2}_{\text{threshold}} \right\} \quad (33)$$

is an asymptotic $(1 - \alpha)$ confidence set that follows the likelihood’s actual shape — asymmetric where the likelihood is asymmetric, respecting parameter bounds by construction.

Each profile point (32) is a constrained re-optimization (L-BFGS-B over the remaining parameters, warm-started at the MLE), so the practical question is *where* to spend those evaluations. The library’s algorithm, per parameter:

1. **Adaptive outward search.** Starting from the MLE with step $\max(0.01|\hat{\theta}_r|, 10^{-4})$, walk right, doubling the step each time, evaluating ℓ_p at each point, until the profile drops below the threshold in (33), a parameter bound is hit (evaluate just inside it), or 15 steps are exhausted. Repeat leftward. Geometric doubling reaches a crossing k standard errors away in $O(\log k)$ evaluations — important because parameters like p can have profiles an order of magnitude wider than their Wald SE suggests.
2. **Infill.** If fewer than `n_points` evaluations accumulated, bisect the largest remaining gap between consecutive evaluated points, repeatedly — concentrating resolution where the profile is least known.
3. **Interval endpoints by interpolation.** The CI endpoints are where ℓ_p crosses the threshold: found by linear interpolation between the bracketing evaluations on each side. If the profile never drops below the threshold before a bound (e.g. p against 0), the interval is truncated at the searched range — reproducing the one-sided behavior a boundary parameter should have.
4. **Standard error from local curvature.** Near the MLE, $\ell_p(v) \approx \ell(\hat{\vartheta}) - (v - \hat{\theta}_r)^2/(2\text{SE}^2)$: a parabola whose leading coefficient a satisfies $\text{SE} = \sqrt{-1/(2a)}$. The code fits that parabola by least squares through the (up to five) highest-likelihood profile points. If the fit is not concave ($a \geq 0$, a genuinely non-quadratic profile), the fallback derives the SE from the interval width instead, $\text{SE} \approx (\text{CI}_{\text{high}} - \text{CI}_{\text{low}})/(2z_{1-\alpha/2})$ — and only when the endpoints genuinely crossed the threshold rather than being truncated at the searched range.

```

1 def optimize_profile(val):
2     def neg_log_lik_profile(free_params):
3         full_params = np.zeros(len(self._param_names))
4         full_params[idx] = val           # parameter of interest,
5         pinned
6         free_idx = 0
7         for j in range(len(self._param_names)):
8             if j != idx:
9                 full_params[j] = free_params[free_idx]
10                free_idx += 1
11        return self.log_likelihood(full_params)
12
13    res = minimize(fun=neg_log_lik_profile, x0=free_init,
14                  method="L-BFGS-B", bounds=free_bounds,
15                  options={"maxiter": 100, "ftol": 1e-6})
16    return -res.fun if res.success and np.isfinite(res.fun) else
17           -np.inf

```

Listing 12: The profiled objective: pin θ_r , optimize the rest (excerpt).

The companion method `plot_profiles()` draws each profile with its threshold line — the visual version of (33), and in practice the fastest way to *see* identifiability problems: a flat profile is a parameter the data cannot pin down.

6.3 Parametric bootstrap

The third route drops the asymptotics altogether and pays with computation. Treat the fitted model as the truth; simulate B datasets of the same length n and spacing Δt from it; re-estimate ϑ on each; and read the sampling distribution of the estimator directly off the B replicate estimates $\hat{\vartheta}^{(1)}, \dots, \hat{\vartheta}^{(B)}$ (Davison and Hinkley, 1997):

$$\widehat{\text{SE}}_r = \text{sd}(\hat{\theta}_r^{(b)}), \quad \text{CI}_{1-\alpha} = \left[\hat{\theta}_r^{(\lceil B\alpha/2 \rceil)}, \hat{\theta}_r^{(\lceil B(1-\alpha/2) \rceil)} \right] \quad (\text{percentile interval}). \quad (34)$$

No normality, no quadratic approximation, and boundary effects are represented honestly: if $\hat{p}$ piles up near zero across replicates, the percentile interval shows it.

Implementation decisions worth recording: replicate b is simulated with seed `seed + b`, making the entire bootstrap reproducible from one integer while keeping replicates independent; re-fits that fail to converge are discarded (their count is reported as `n_successful`), since averaging over failed optimizations would contaminate (34) with optimizer artifacts; the SE uses the `ddof = 1` sample standard deviation; and the raw replicate matrix is stored in the results dictionary so users can compute BCa or basic intervals, correlations between parameters, or replicate diagnostics without re-running the loop.

```

1 model = JumpDiffusionModel(jump_distribution=jump_dist,
2                             **fitted_params)
3 for b in range(n_bootstrap):
4     rep_seed = None if seed is None else seed + b
5     _, path, _ = model.simulate(T=horizon, n_steps=n_steps, x0=0.0,
6                                seed=rep_seed)
7     increments = np.diff(path)

```

```

7     estimator = JumpDiffusionEstimator(increments, self.dt,
8                                         jump_distribution=jump_dist)
9     rep_result = estimator.estimate(**estimate_kwargs)
10    if rep_result["convergence"]:
11        replicates.append([rep_result["parameters"][name]
12                           for name in self._param_names])

```

Listing 13: The bootstrap loop (excerpt).

All three routes converge into one view: `summary()` returns a tidy table with one row per parameter and one column block per computed route, and `diagnostics()` prints it with convergence and data statistics — making the disagreement between routes (the interesting part) visible at a glance.

`diagnostics()` also compares the sample mean and variance of the increments against the ones the fitted model *implies*. Under the Bernoulli mixture (5) these are exact:

$$\mathbb{E}[\Delta X] = \mu \Delta t + p \mathbb{E}[J], \quad \text{Var}[\Delta X] = \sigma^2 \Delta t + p \mathbb{E}[J^2] - (p \mathbb{E}[J])^2, \quad (35)$$

with $\mathbb{E}[J]$ and $\text{Var}[J]$ supplied by the jump law’s generic numerical moments (Section 3.1).

Remark 6.1 (The jump term in the mean is not optional). An earlier version of `diagnostics()` printed $\mu \Delta t$ alone as the theoretical mean. With symmetric jumps that omission is invisible ($\mathbb{E}[J] = 0$), but the 2026 audit measured its effect on a skew-normal fit: a reported theoretical mean of -0.000019 against an empirical mean of 0.011798 — suggesting a gross model–data mismatch — when the model’s actual implied mean, $\mu \Delta t + p \mathbb{E}[J] = 0.011778$, agreed with the data almost exactly. For *asymmetric* jump laws, the whole point of this library, the $p \mathbb{E}[J]$ term routinely dominates $\mu \Delta t$; equation (35) is now computed in full.

7 Testing for the presence of jumps

Before interpreting fitted jump parameters, one should ask whether the data support jumps at all:

$$H_0 : p = 0 \quad \text{vs.} \quad H_1 : p > 0. \quad (36)$$

The natural statistic is the likelihood ratio. Under H_0 the model collapses to pure diffusion, whose increments are i.i.d. $\mathcal{N}(\mu \Delta t, \sigma^2 \Delta t)$ — so the null MLE is available in closed form (sample mean and MLE standard deviation of the increments, no optimizer needed) and

$$\text{LR} = 2 \left[\ell(\hat{\vartheta}) - \ell_0(\hat{m}, \hat{s}) \right], \quad \ell_0(\hat{m}, \hat{s}) = \sum_{i=1}^n \log \phi(\Delta x_i; \hat{m}, \hat{s}^2). \quad (37)$$

7.1 Why the textbook calibration fails twice

Were this a regular testing problem, $\text{LR} \rightsquigarrow \chi_d^2$ with d the number of restricted parameters. Here two pathologies invalidate that, *simultaneously*:

(i) The null is on the boundary. $p = 0$ is not an interior point of $[0, 1]$. When a tested parameter sits on the boundary of the parameter space, the LR statistic’s limit is not χ^2 but a *mixture* — in the simplest case $\frac{1}{2}\chi_0^2 + \frac{1}{2}\chi_1^2$, reflecting that with probability one half the unconstrained estimate falls on the wrong side of the boundary and the statistic degenerates to zero (Chernoff, 1954; Self and Liang, 1987).

(ii) **Nuisance parameters vanish under the null.** When $p = 0$, the jump-size parameters θ_J multiply nothing: the likelihood (6) is flat in θ_J , so they are *unidentified* under H_0 . This is Davies’ problem (Davies, 1977, 1987): the LR statistic is then the supremum of a process indexed by the unidentified parameters, its null distribution depends on the model and the data design, and no universal table exists.

Each pathology alone has known corrections; their combination, for this mixture with five different jump families, does not admit a practical closed-form calibration.

7.2 The parametric bootstrap test

Design decision 7.1 (Simulate the null distribution instead of assuming one). The library sidesteps both pathologies with a Monte Carlo test (Davison and Hinkley, 1997): whatever the true null distribution of LR is — boundary mixture, Davies supremum, finite-sample quirks and all — it can be sampled from, because the null model is fully specified once fitted. The procedure:

1. Fit the null on the observed data: $\hat{m}, \hat{s}$ in closed form; compute the observed LR via (37).
2. For $b = 1, \dots, B$: simulate n increments i.i.d. $\mathcal{N}(\hat{m}, \hat{s}^2)$; fit the *full* jump-diffusion model on them; compute $\text{LR}^{(b)}$ the same way.
3. Report the Monte Carlo p-value

$$\hat{p}_{\text{MC}} = \frac{1 + \#\{b : \text{LR}^{(b)} \geq \text{LR}\}}{B_{\text{succ}} + 1}, \quad (38)$$

where B_{succ} counts the replicates whose full-model re-fit converged.

The “+1” in numerator and denominator makes (38) an exact-level p-value: it treats the observed statistic as one more draw from the null, which is precisely what it is when H_0 holds (Davison and Hinkley, 1997). The price is B full model fits — the same cost profile as the bootstrap intervals of Section 6.3.

Two implementation details guard the statistic itself. First, the full model nests the null, so in exact arithmetic $\text{LR} \geq 0$; in practice the optimizer can stop a hair short of the null likelihood, so the code clamps the statistic at zero — on both the observed data and every replicate, keeping the comparison in (38) consistent. Second, the null MLE uses the population ($\text{ddof} = 0$) standard deviation: that is the Gaussian *maximum likelihood* estimate, and using the unbiased ($\text{ddof} = 1$) version would slightly misstate ℓ_0 and bias the statistic.

```

1 log_lik_full = results["log_likelihood"]
2 log_lik_null, mean0, std0 =
    self._pure_diffusion_log_likelihood(self.increments)
3 # The full model nests the null, so LR is non-negative in theory;
4 # clamp to guard against the optimizer stopping just short of it.
5 lr_observed = max(0.0, 2.0 * (log_lik_full - log_lik_null))
6
7 rng = np.random.default_rng(seed)
8 bootstrap_stats = []
9 for _ in range(n_bootstrap):
10     increments_b = rng.normal(mean0, std0, size=self.n_obs)
11     estimator = JumpDiffusionEstimator(increments_b, self.dt,
12                                         jump_distribution=jump_dist)
```

```

13     rep_result = estimator.estimate(**estimate_kwargs)
14     if not rep_result["convergence"]:
15         continue
16     log_lik_full_b = rep_result["log_likelihood"]
17     log_lik_null_b, _, _ =
18         self._pure_diffusion_log_likelihood(increments_b)
19     bootstrap_stats.append(max(0.0, 2.0 * (log_lik_full_b -
20         log_lik_null_b)))
21
22 exceed = int(np.sum(np.asarray(bootstrap_stats) >= lr_observed))
23 p_value = (1.0 + exceed) / (len(bootstrap_stats) + 1.0)

```

Listing 14: The jump test in `estimation/maximum_likelihood.py` (excerpt).

Remark 7.1 (What remains to be established). The bootstrap test is valid by construction *under the fitted null*; its finite-sample size across sample lengths and its power against alternatives with small p or jump scales close to the diffusion scale are empirical questions — precisely the ones the Monte Carlo study of Section 9 is designed to answer, with the naive χ^2 and the $\frac{1}{2}-\frac{1}{2}$ mixture calibrations as comparators.

8 Comparing jump distributions and validating the estimator

8.1 Ranking candidate jump laws

With five interchangeable jump families, the applied question becomes: which one does *this* dataset support? `JumpDistributionComparison` fits each candidate to the same increments and reports two complementary kinds of evidence.

Information criteria. AIC and BIC from (28) penalize the maximized likelihood for parameter count — relevant here because the candidates differ in dimension (Merton normal has one jump parameter, the SGED four). They answer a *relative* question: which candidate trades fit against complexity best.

Goodness of fit. Information criteria cannot say whether even the best candidate actually fits. For that, the comparison computes a Kolmogorov–Smirnov distance between the observed increments and the fitted model’s increment distribution — the latter having no closed-form CDF, it is represented by a large simulated reference sample from the fitted model, so the statistic is a two-sample KS distance whose reference-side noise is made negligible by sample size.

Design decision 8.1 (A bootstrap p-value, because parameters were estimated). The classical KS p-value assumes the reference distribution was fixed *a priori*. Here it was estimated from the very data being tested, which makes the naive p-value strongly anti-conservative — the fitted model always looks too good, the same phenomenon that motivates the Lilliefors correction for Gaussian fits. The library restores validity with the same device as Section 7: simulate B datasets from the fitted model, *re-fit the model on each* (re-fitting is what accounts for estimation error), recompute each replicate’s KS distance to its own re-fitted model, and report the Monte Carlo p-value (38). A model is inadequate when the observed distance is large relative to the distances the model produces against itself.

```

1 for b in range(n_bootstrap):
2     boot_data = self._simulate_increments(fitted_model, n_obs,
3         data_seed)
4     estimator = JumpDiffusionEstimator(
5         boot_data, self.dt, jump_distribution=jump_distribution)
6     rep_result = estimator.estimate(**estimate_kwargs)
7     if not rep_result["convergence"]:
8         continue
9     model_b = JumpDiffusionModel(jump_distribution=jump_distribution,
10        **rep_result["parameters"])
11    reference_b = self._simulate_increments(model_b, reference_size,
12        ref_seed)
13    ks_b = float(ks_2samp(boot_data, reference_b).statistic)
14    if ks_b >= ks_observed:
15        exceed += 1
16    n_successful += 1
17
18 return (1.0 + exceed) / (n_successful + 1.0)

```

Listing 15: Bootstrap KS p-value in `validation/distribution_comparison.py` (excerpt).

`compare()` assembles everything into a table ranked by AIC (with BIC, KS statistic and bootstrap p-value alongside), and `plot_comparison()` overlays each fitted mixture density on the increment histogram — the visual complement to the numbers. Because the KS machinery relies only on simulation through `JumpDistribution.rvs`, it works unchanged for any user-added jump family, honoring the extension contract of Section 3.1.

8.2 Monte Carlo validation of the estimator

Finally, `ValidationExperiment` closes the loop on the estimator itself: choose true parameters, simulate R independent paths, estimate on each, and summarize the sampling behavior of the estimator — bias, standard deviation, RMSE per parameter, along with convergence rates (all summary statistics condition on the runs whose optimization converged). Two reporting conventions are deliberate. Relative errors are reported as NaN whenever the true parameter is (numerically) zero — $\mu = 0$, `jump_skew` = 0 and `jump_loc` = 0 are all natural experimental settings, and division by zero would otherwise inject $\pm\infty$ into every downstream summary; NaN says “undefined”, which is the honest answer. And the share of runs landing within 5% of the truth is reported as an *accuracy rate* (`within_5pct_rate`), pointedly not called “coverage”: coverage belongs to the confidence intervals of Section 6, and conflating the two invites exactly the kind of misreading a validation tool exists to prevent.

This simulate–estimate–compare pattern is how every method in this paper was tested during development, packaged for users; it is also the seed of the full factorial Monte Carlo study (coverage of the three interval routes, size and power of the jump test, identifiability maps) that constitutes the research program around the library.

9 Closing remarks and research agenda

This paper has walked through the full pipeline implemented by `jump-diffusion-estimation`: an exactly-solvable SDE whose only approximation is the Bernoulli treatment of the jump count

(Section 2); five jump families behind a density-only extension contract, with closed-form Gaussian convolutions exploited wherever the mathematics allows (Section 3) and an FFT scheme, ported from Ospina Arango (2009), wherever it does not (Section 4); maximum likelihood driven by either a local quasi-Newton method or the differential evolution configuration whose robustness was the thesis’ applied finding (Section 5); three inference routes of increasing robustness and cost (Section 6); a parametric-bootstrap likelihood-ratio test that sidesteps the boundary and Davies pathologies by simulating the null distribution outright (Section 7); and estimation-error-aware goodness-of-fit comparison across jump families (Section 8).

A recurring pattern deserves to be named, because it *is* the library’s design philosophy:

- **Closed forms first, generic numerics as fallback.** Every numerical fallback (FFT convolution, inverse-CDF sampling) is validated against the closed forms available in special cases.
- **Pathologies are mapped to explicit poison values** — $+\infty$ objectives, NaN standard errors — never silently absorbed.
- **When asymptotics are doubtful, simulate.** The bootstrap appears three times (intervals, jump test, KS p-value), each time replacing a questionable asymptotic calibration with an honest Monte Carlo one.
- **Reproducibility is an argument, not an accident:** every stochastic routine accepts a seed and derives per-replicate seeds deterministically from it.

Open questions

The library makes a set of methods available; it does not yet certify their finite-sample behavior. The companion research program, for which this library is the experimental engine, targets exactly that gap:

1. **Coverage.** Empirical coverage of the Wald, profile and bootstrap intervals across jump families, sample sizes, jump intensities and jump-to-diffusion scale ratios — when does profiling recover the nominal coverage that Wald loses?
2. **Jump test calibration.** Size and power of the bootstrap LR test of Section 7 against the naive χ^2 and the boundary-corrected mixture calibrations.
3. **Identifiability.** Mapping the region of $(\lambda\Delta t, \text{jump scale}/\sigma\sqrt{\Delta t})$ where p , σ and the jump scale can be jointly recovered, via the conditioning of the observed information and the sampling correlation of $(\hat{p}, \hat{\sigma})$.
4. **Bernoulli error.** Quantifying the bias of Decision 2.1 against the exact Poisson mixture as $\lambda\Delta t$ grows.
5. **Optimizer geography.** Where in parameter space multistart L-BFGS-B fails and differential evolution succeeds, extending Ardia et al. (2011) from a demonstration to a map.

Each question already has its instrument in the library; what remains is the systematic experiment. That study is deliberately kept out of this document: here we fixed *what the code computes and why*, so that the experiments — and any reader’s own use of the library — stand on explicit mathematical ground.

References

- David Ardia, Juan David Ospina Arango, and Norman Diego Giraldo Gómez. Jump-diffusion calibration using differential evolution. *Wilmott*, 2011(55):76–79, 2011. doi: 10.1002/wilm.10034.
- Adelchi Azzalini. A class of distributions which includes the normal ones. *Scandinavian Journal of Statistics*, 12(2):171–178, 1985.
- Adelchi Azzalini. Further results on a class of distributions which includes the normal ones. *Statistica*, 46(2):199–208, 1986.
- Tim Bollerslev. A conditionally heteroskedastic time series model for speculative prices and rates of return. *The Review of Economics and Statistics*, 69(3):542–547, 1987. doi: 10.2307/1925546.
- Herman Chernoff. On the distribution of the likelihood ratio. *The Annals of Mathematical Statistics*, 25(3):573–578, 1954. doi: 10.1214/aoms/1177728725.
- Robert B. Davies. Hypothesis testing when a nuisance parameter is present only under the alternative. *Biometrika*, 64(2):247–254, 1977. doi: 10.1093/biomet/64.2.247.
- Robert B. Davies. Hypothesis testing when a nuisance parameter is present only under the alternative. *Biometrika*, 74(1):33–43, 1987. doi: 10.1093/biomet/74.1.33.
- Anthony C. Davison and David V. Hinkley. *Bootstrap Methods and their Application*. Cambridge University Press, Cambridge, 1997. doi: 10.1017/CBO9780511802843.
- Bradley Efron and David V. Hinkley. Assessing the accuracy of the maximum likelihood estimator: Observed versus expected Fisher information. *Biometrika*, 65(3):457–483, 1978. doi: 10.1093/biomet/65.3.457.
- Carmen Fernández and Mark F. J. Steel. On Bayesian modeling of fat tails and skewness. *Journal of the American Statistical Association*, 93(441):359–371, 1998. doi: 10.1080/01621459.1998.10474117.
- Steven G. Kou. A jump-diffusion model for option pricing. *Management Science*, 48(8):1086–1101, 2002. doi: 10.1287/mnsc.48.8.1086.166.
- Robert C. Merton. Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1–2):125–144, 1976. doi: 10.1016/0304-405X(76)90022-2.
- Juan David Ospina Arango. Estimación de un modelo de difusión con saltos con distribución de error generalizada asimétrica usando algoritmos evolutivos. Master’s thesis, Universidad Nacional de Colombia, Medellín, Colombia, 2009.
- Steven G. Self and Kung-Yee Liang. Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *Journal of the American Statistical Association*, 82(398):605–610, 1987. doi: 10.1080/01621459.1987.10478472.
- Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.

Panayiotis Theodossiou. Skewed generalized error distribution of financial assets and option pricing. *Multinational Finance Journal*, 19(4):223–266, 2015. doi: 10.17578/19-4-1.

Samuel S. Wilks. The large-sample distribution of the likelihood ratio for testing composite hypotheses. *The Annals of Mathematical Statistics*, 9(1):60–62, 1938. doi: 10.1214/aoms/1177732360.